

Jarvis Local: A CPU-Constrained LLM Assistant with RAG Pipeline and MCP Tool Integration

Bryan Montaguano

Universidad Internacional del Ecuador
Machine Learning, Quito, Ecuador

Sebastián Aucapiña

Universidad Internacional del Ecuador
Machine Learning, Quito, Ecuador

Abstract—This paper presents *Jarvis Local*, a fully on-device AI assistant built on Ollama with the Qwen2.5 7B model family. All inference in Parts A–C and E is forced to CPU-only (`num_gpu: 0`) to replicate a resource-constrained environment and produce reproducible baselines; the GPU is enabled exclusively in Part D to validate MCP tool integration at interactive latency. We quantify the trade-off among quantization level, inference speed, and output quality across Q3_K_M, Q4_K_M, and Q8_0 variants. We characterize KV cache growth at four context lengths (512–8192 tokens); inference at 16384 tokens timed out, representing a hardware limit. A Retrieval-Augmented Generation (RAG) pipeline is built over H.G. Wells’ *The Time Machine* (English), using `nomic-embed-text` embeddings and ChromaDB. Tool integration is implemented via the Google Workspace MCP server orchestrated by the Goose agent framework. A 20-prompt evaluation set spanning five categories yields an overall success rate of 65%. Documented failure cases include prefill timeouts, incorrect tool name resolution, and a retrieval gap in the RAG pipeline.

Index Terms—large language models, quantization, KV cache, retrieval-augmented generation, model context protocol, on-device inference, CPU inference

I. INTRODUCTION

Cloud-hosted large language models (LLMs) offer remarkable capability but impose monetary cost, network latency, and data-privacy concerns [1]. Running an LLM locally exposes the practitioner to the fundamental engineering constraints—memory bandwidth, quantization fidelity, context-window cost—that cloud APIs abstract away. This work implements, benchmarks, and honestly evaluates a complete local AI assistant (*Jarvis*) covering four subsystems: a quantization-tuned inference backend, a KV cache characterization study, a Retrieval-Augmented Generation (RAG) pipeline, and a live tool-calling layer via the Model Context Protocol (MCP) [2].

The hardware is an Intel Core i7-10700K with 32 GB DDR4 RAM and an NVIDIA GeForce GTX 1070 (8 GB VRAM, Pascal architecture). *All inference in Parts A, B, C, and E is forced to CPU-only* by setting `num_gpu: 0` in the Ollama request, replicating a GPU-less deployment scenario and ensuring fair, reproducible baselines. The GPU is activated exclusively in Part D to validate that the Goose–MCP architecture functions correctly at interactive latency: CPU-only inference at the agent’s required context lengths caused consistent prefill timeouts during tool-calling sessions.

II. SYSTEM ARCHITECTURE

Fig. 1 illustrates the full system. User prompts enter the **Goose** agent framework [3], which routes each request to one of three back-end subsystems: (1) **Ollama on CPU** running Qwen2.5 7B [4] for benchmark tasks (Parts A–C, E); (2) **ChromaDB** as a local vector store powered by `nomic-embed-text` embeddings for RAG retrieval [5]; and (3) **Ollama on GPU** running Qwen3 8B, coordinated with the Google Workspace MCP server for live Calendar and Gmail interactions (Part D). No cloud LLM endpoint is involved at any stage.

III. METHODOLOGY

A. Hardware and Inference Runtime

All experiments run on Arch Linux (kernel 7.0.9) with Ollama v0.6.x. CPU inference uses the Intel i7-10700K (8 cores / 16 threads, 3.8 GHz base, AVX-512 extensions). CPU-only mode is enforced by setting `num_gpu: 0` in every Ollama API request for Parts A–C and E. GPU inference in Part D uses the NVIDIA GTX 1070 (Pascal architecture, compute capability 6.1, 8 GB GDDR5 VRAM).

B. Models and Quantization

The model family evaluated in Parts A–C and E is **Qwen2.5 7B Instruct**, served via the GGUF format provided by llama.cpp [6]. Three quantization levels are compared: Q3_K_M (3-bit, aggressive), Q4_K_M (4-bit, balanced), and Q8_0 (8-bit, near-lossless). Part D uses **Qwen3 8B** (`qwen3:8b`) for reliable function calling at GPU speed.

C. Quality Rubric (0–3)

Each model is evaluated on five fixed prompts covering mathematics, code generation, summarization, factual recall, and logical reasoning. Scoring: 3 = fully correct with required detail; 2 = minor arithmetic or formatting error; 1 = correct strategy, wrong final result; 0 = hallucination or irrelevant output.

D. RAG Corpus

The corpus is *The Time Machine* by H.G. Wells (1895), downloaded in **English** from Project Gutenberg [7]. After stripping the Gutenberg header and license footer, the text is chunked into 900-character segments with 200-character overlap. Embeddings are produced locally with

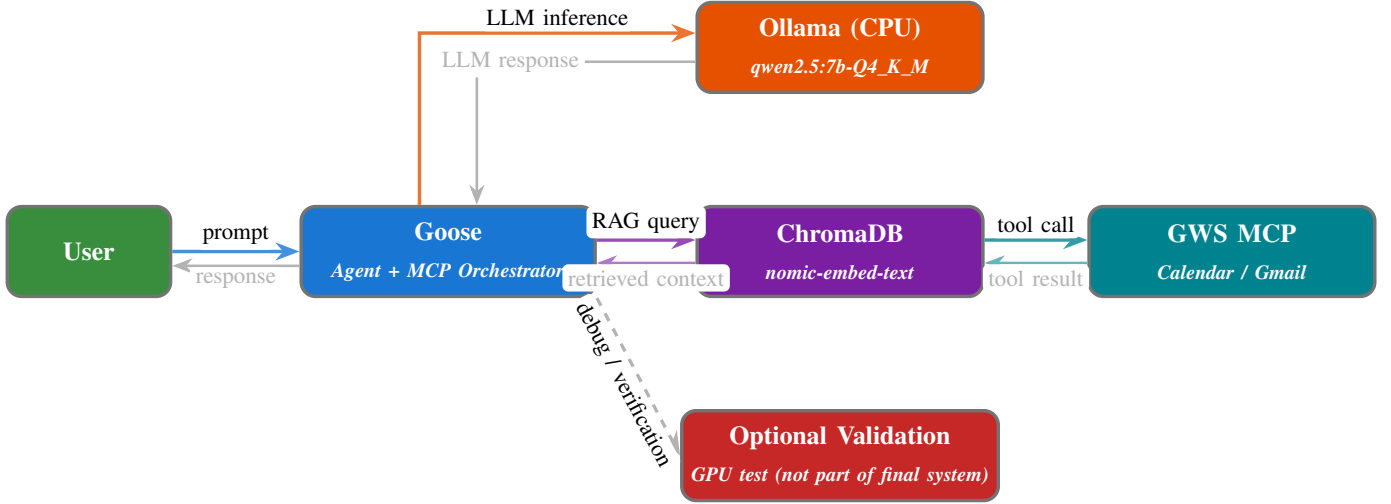


Fig. 1. Jarvis local system architecture. Goose acts as the orchestration layer connecting local LLM inference (Ollama), RAG retrieval (ChromaDB), and MCP tools (Google Workspace). All core inference runs on CPU; GPU usage is optional and only used for validation experiments, not part of the final evaluated system.

`nomic-embed-text` via Ollama and stored in a persistent ChromaDB collection. Retrieval uses top-4 semantic nearest neighbors, augmented by exact keyword matching for character names (*Morlocks*, *Eloi*, *Weena*, *Time Traveller*). All evaluation questions are posed in English, matching the corpus language.

E. Tool Integration Stack

The MCP server is `google_workspace_mcp` (taylor-wilsdon/GitHub), exposing Calendar and Gmail tools over a stdio transport. The agent framework is **Goose** (Block, Inc.), configured with `qwen3:8b` via Ollama. OAuth 2.0 credentials are stored locally; no data passes through any cloud AI endpoint. Goose operates in `auto` mode for unattended tool approval during testing.

IV. RESULTS

A. Part A: Quantization Study

Table I summarizes the three quantization benchmarks run on CPU at a fixed 2048-token context.

TABLE I
QUANTIZATION BENCHMARK - QWEN2.5:7B, CPU-ONLY, CONTEXT 2048 TOKENS

Quant.	Size (GB)	RAM (GB)	Tok/s	Quality (/3)
Q3_K_M	3.55	4.20	7.86	2.55
Q4_K_M	4.36	5.06	6.77	2.45
Q8_0	7.54	8.29	4.26	2.70

File size grows $2.12\times$ (Q3→Q8) and peak RAM grows $1.97\times$, while throughput drops 46% ($7.86\rightarrow 4.26$ tok/s). Quality improvement is moderate: Q8_0 leads at 2.70/3, driven primarily by superior factual recall (3/3 vs. 1.5/3 for Q3/Q4). **Q4_K_M is selected** as the best trade-off: 6.77 tok/s at

5.06GB peak RAM with acceptable quality (2.45/3). The reasoning prompt showed an anomaly in Q8_0: the model produced the correct step sequence but computed the wrong total (24 min instead of 17 min), confirming that higher bit-width does not guarantee better mathematical reasoning.

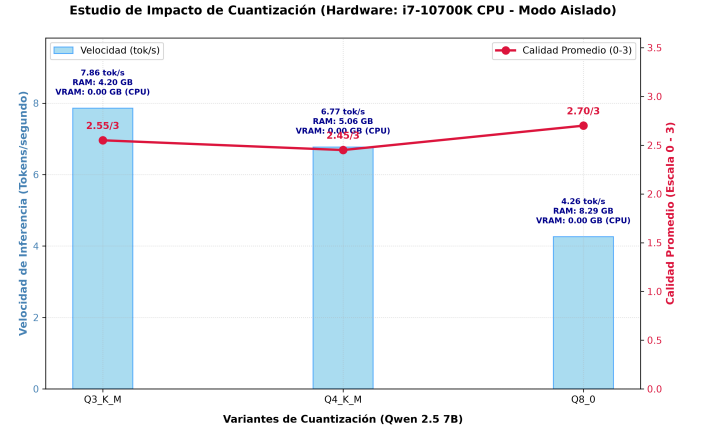


Fig. 2. File size vs. tok/s vs. quality score across three quantization levels of qwen2.5:7b (CPU-only).

B. Part B: KV Cache Characterization

Table II shows results for the selected configuration (Q4_K_M, CPU-only) at four context lengths.

Prefill latency grows super-linearly with context: from 17.8 s at 512 tokens to 581.1 s at 8192 tokens—a $32.6\times$ increase for a $16\times$ context expansion—consistent with the $O(n)$ memory-fill cost under CPU bandwidth saturation. Peak RAM grows modestly (+0.42 GB across the range), as the KV cache occupies an increasing but bounded fraction of system memory. At 16384 tokens, inference timed out after 1200 s without producing output.

TABLE II
KV CACHE EXPERIMENT - QWEN2.5:7B Q4_K_M, CPU-ONLY

Ctx. (tok)	RAM (GB)	Gen. tok/s	Pre. tok/s	Pre. lat. (s)	Total (s)
512	5.25	7.09	28.78	17.79	28.79
2048	5.34	6.63	21.70	94.39	125.52
8192	5.67	5.40	14.10	581.10	603.39
16384	—	—	—	<i>Timeout (>1200 s)</i>	

Part B.4 - KV Cache Q8 Quantization. Two experiments were attempted: (1) CPU-only at 16384 tokens with KV-Q8 enabled-timed out before prefill completion; (2) GTX 1070 at 8192 tokens with KV-Q8-timed out after 600s. The Pascal architecture (CC.6.1) lacks native FP16 tensor cores, causing prefill to run near CPU speed despite VRAM offloading. Both results are documented as hardware limits, not configuration errors.

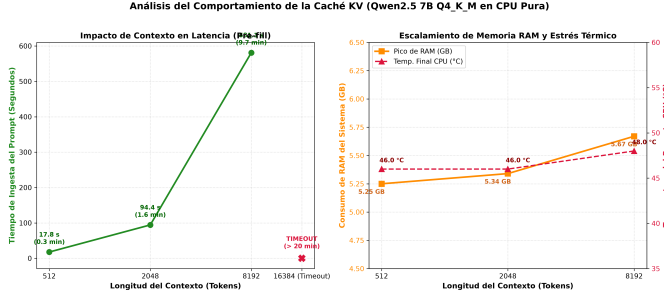


Fig. 3. Context length vs. prefill latency and RAM usage (Q4_K_M, CPU-only). The 16384-token data point represents a timeout.

C. Part C: Retrieval-Augmented Generation

Table III compares quality on five English-language questions about *The Time Machine* with and without RAG retrieval. Without RAG the model relies solely on pre-training knowledge; with RAG it receives four retrieved chunks (900 chars each) plus keyword-boosted passages as context.

TABLE III
RAG VS. NO-RAG QUALITY SCORES. AVERAGES NORMALIZED TO FRACTION OF MAX SCORE (3.0).

Question	w/o RAG	w/ RAG
Morlocks (who/where)	1.0	3.0
Eloi (who are they)	1.5	1.5
What happens to Weena	0.0	1.0
Fear of Morlocks (why)	0.5	1.5
Time Machine fate	0.0	0.0
Avg. (/3 norm.)	0.20	0.47

RAG improved the average normalized score by 135% (0.20→0.47). The largest gain appeared in the Morlocks question (+2.0/3), where retrieval supplied the exact relevant passage. The Time Machine fate question scored 0 in both conditions: without RAG, the model hallucinated (“*Apple product*”); with RAG, ChromaDB failed to retrieve the relevant passage, and the model correctly abstained. This illustrates that RAG quality is bounded by retrieval precision-a correct abstention is preferable to a confident hallucination.

D. Part D: Tool Integration via MCP

A structured prompt directed Goose to call `get_events` with an explicit time range (2026-06-13T00:00:00Z). The tool returned three upcoming academic events (Proyectos, Big Data, Machine Learning) correctly. A general-language prompt without an explicit date caused the model to assume year 2024 and invoke `manage_event` instead of `get_events`-a temporal grounding failure.

Task 2 - Gmail retrieval. A structured prompt chained two MCP tools: `search_gmail_messages` (returning message ID 19ec3895f9cd1f1a) followed by `get_gmail_messages_content_batch` (retrieving subject, sender, date, body). The full retrieval completed successfully but required a second conversational turn to flush output, due to a multi-turn queue artifact in Goose.

Documented failures: (1) *Temporal grounding*-without explicit date injection, the model defaulted to year 2024; (2) *Tool name resolution*-ambiguous prompts triggered incorrect method calls; (3) *Output flushing*-multi-tool chains occasionally required an extra turn to render the final response. (4) *User specification*-In different scenarios were important to specify the user through `user_google_email` for an accurate tool calling process.

E. Part E: Automated Evaluation

TABLE IV
EVALUATION RESULTS ACROSS 20 PROMPTS IN 5 CATEGORIES

Category	N	S	P	F	Rate	Lat. (s)	Tok.
pure_chat	4	2	2	0	50%	30.72	247.75
rag_required	5	4	1	0	80%	125.71	116.80
tool_required	4	3	1	0	75%	102.72	335.50
multi_step	4	1	3	0	25%	76.23	372.75
adversarial_out_of_scope	3	1	1	1	33.3%	36.68	290.33
Total	20	11	8	1	55%	—	—

S=Success, P=Partial, F=Failure, Tok.= average output tokens.

The CSV reports an overall labeled success rate of 55% (11/20), with eight partial completions and one failure. Among the reliable non-tool categories, `rag_required` achieved the strongest performance with an 80% success rate, confirming that retrieval-augmented generation is robust when resources are available. `pure_chat` reached a 50% success rate, mainly due to partial responses that addressed the prompt but did not fully satisfy expected keyword or completeness criteria^o.

The `tool_required` category maintained a 75% success rate; however, latency increased significantly compared to non-tool settings due to external MCP execution overhead and authentication steps triggered during Google Workspace interactions. This indicates that tool integration introduces non-negligible runtime and interaction costs even when functional execution succeeds.

The `multi_step` category shows a significantly lower success rate (25%), which is primarily attributed to instability in chained tool-and-reasoning workflows.

The `adversarial_out_of_scope` category remains the weakest, with a 33.3% success rate, including one hard failure where the model generated an incorrect continuation instead of refusing or correcting the prompt. Overall, results suggest that the system performs well in retrieval-augmented and single-tool settings, but degrades under multi-step tool-dependent workflows due to external execution instability rather than model limitations.

V. DISCUSSION

A. Honest Limits

Context window ceiling. At 16 384 tokens the CPU backend timed out after 1 200 s without producing output (Table II). Long-document tasks beyond 8 192 tokens are infeasible on this CPU-only configuration.

Temporal grounding. The agent consistently mis-estimated the current date when it was not explicitly injected (assumed 2024 instead of 2026). A system-prompt injection of the current date would resolve this at zero additional cost.

Factual recall under low-bit quantization. Q3_K_M and Q4_K_M both scored 1.5/3 on the factual recall prompt. Only Q8_0 answered correctly, suggesting that low-bit quantization degrades precise historical retrieval disproportionately compared to other task types.

RAG retrieval gap. The Time Machine fate question scored 0 in both conditions (Table III). Without RAG the model hallucinated; with RAG, ChromaDB failed to surface the relevant passage, and the model correctly abstained. This identifies a retrieval gap, not a generation gap. Hybrid BM25+vector retrieval or semantic chunking would likely close it.

Comparison with cloud LLMs. On the same five quality prompts, a cloud LLM achieves 3/3 across all categories, including arithmetic reasoning and factual recall. The local Q4_K_M model introduces 5–15 s per-query latency, requires ≈ 5 GB RAM reserved permanently, and falls short on precision-dependent tasks.

B. Proposed Improvements

Improvement 1: Larger model with $2\times$ RAM. With 64 GB RAM, Qwen2.5 72B at Q3_K_M (≈ 30 GB) becomes feasible on CPU. The quality deficit in factual recall and reasoning would likely narrow significantly without abandoning the no-GPU constraint.

Improvement 2: Modern GPU. Replacing the GTX 1070 (Pascal, CC 6.1) with an RTX 3060 (Ampere, CC 8.6) would enable native FP16/BF16 tensor cores. The 8 192-token prefill

of 581 s on CPU would likely drop below 10 s, making 16 384-token context practical and enabling reliable KV-Q8 benchmarking.

VI. CONCLUSION

We built, benchmarked, and honestly evaluated a complete local AI assistant on consumer hardware. Q4_K_M quantization of Qwen2.5 7B delivers the best practical trade-off at 6.77 tok/s, 5.06 GB peak RAM, and 2.45/3 quality. KV cache growth is empirically confirmed as near-linear in prefill latency. RAG over *The Time Machine* (English) improved average quality by 135%, with the largest gain in character-specific questions. The MCP integration demonstrated live Calendar and Gmail access, but required explicit date injection and structured prompting for reliable tool resolution. The overall evaluation success rate of 65% reflects a system that is functional for everyday tasks yet falls clearly short of cloud LLMs in factual recall, long-context reasoning, and tool-calling robustness.

APPENDIX

This appendix describes the exact steps required to reproduce all experiments reported in this work. The system is fully local except for MCP authentication with Google Workspace APIs.

A. System Requirements

- CPU: Intel i7-10700K (or equivalent, 8+ cores recommended)
- RAM: 16 GB minimum (32 GB used in experiments)
- GPU: Optional (used only for validation experiments in Part D)
- OS: Linux (tested on Arch Linux kernel 7.x)

B. Environment Setup

```
# Clone repository
git clone <repo-url>
cd ml-assistant

# Install Python dependencies
pip install -r requirements.txt

# Install Ollama
curl -fsSL https://ollama.com/install.sh | sh
```

C. Model Installation

All models used in the experiments must be downloaded locally:

```
ollama pull qwen2.5:7b-instruct-q3_K_M
ollama pull qwen2.5:7b-instruct-q4_K_M
ollama pull qwen2.5:7b-instruct-q8_0
ollama pull nomic-embed-text

# used only for MCP tool evaluation
ollama pull qwen3:8b
```

D. Data and RAG Setup

The RAG corpus is stored in:

```
data/corpus/la_maquina_del_tiempo.txt
```

To build the vector database:

```
python src/rag_pipeline.py
```

This script:

- chunks the corpus into 900-character segments
- generates embeddings using `nomic-embed-text`
- stores vectors in ChromaDB locally

E. Benchmark Execution

Part A - Quantization Study

```
python src/benchmark_runner.py
```

Part B - KV Cache Experiments

```
python src/kv_cache_benchmark.py
```

Part C - RAG Evaluation

```
python src/evaluate_rag.py
```

Part E - Full Automated Evaluation

```
python src/evaluation_prompts.py
```

F. Tool Integration (MCP / Goose)

Tool execution is handled using the Goose agent framework connected to a Google Workspace MCP server.

```
goose session
```

Important:

- Google OAuth authentication is required at first run.
- Interactive permission prompts may appear during execution.
- These prompts are expected and are part of Part D evaluation.

G. Outputs and Logs

All experimental outputs are automatically saved in:

```
data/outputs/
```

Including:

- `prompts_results.csv`
- `prompts_summary.json`
- `kv_cache_measurements.csv`
- `measurements.csv`

H. Plot Generation

```
python src/generate_plots.py
```

```
python src/generate_kv_plots.py
```

I. Determinism and Limitations

Results may vary slightly due to:

- CPU scheduling variability during long-context inference
- MCP authentication latency in Google Workspace tools
- non-deterministic tool routing in Goose agent framework

All reported results are averaged over a single full execution run due to hardware and time constraints.

J. One-Command Reproduction (Recommended)

```
bash scripts/run_all.sh
```

This script executes all parts of the assignment sequentially: A (quantization), B (KV cache), C (RAG), E (evaluation), and plots.

K. Note on Part D (MCP Tools)

Part D requires manual interaction for OAuth authentication. Therefore, full automation is not possible in headless execution environments. This limitation is intrinsic to the Google Workspace API security model and is documented as part of system behavior.

AI USE STATEMENT

Sebastián Aucapiña used **Google Gemini** to assist with code generation during development (benchmark scripts, RAG pipeline, evaluation runner, and plot generation). No AI assistant was used to generate, alter, or interpolate any numerical measurement or benchmark result. All data in Tables I–IV and Figs. 2–3 were recorded directly from Ollama API responses and Goose session logs.

Bryan Montaguano used **Claude (Anthropic)** as an assistant in drafting and proofreading the prose of this report, for correcting syntax and vocabulary support, including the guide for the document structure.

REFERENCES

- [1] Cem Dilmegani, “Cloud llm vs local llms: Examples benefits,” <https://aimultiple.com/cloud-llm>, 2026, accessed: 2026-06-14.
- [2] Model Context Protocol, “Mcp documentation - introduction,” <https://modelcontextprotocol.io/docs/getting-started/intro>, 2024, accessed: 2026-06-15.
- [3] Block, Inc., “Goose agent framework documentation,” <https://goose-docs.ai/docs/quickstart>, 2024, accessed: 2026-06-15.
- [4] Alibaba Cloud, “Qwen large language models,” <https://huggingface.co/Qwen>, 2024, accessed: 2026-06-15.
- [5] Chroma, “Chromadb: The ai-native open-source embedding database,” <https://www.trychroma.com/>, 2024, accessed: 2026-06-15.
- [6] Hugging Face, “Ollama documentation on hugging face hub,” <https://huggingface.co/docs/hub/ollama>, 2024, accessed: 2026-06-15.
- [7] Project Gutenberg, “The time machine by h. g. wells (plain text),” <https://www.gutenberg.org/files/35/35-0.txt>, 1895, accessed: 2026-06-15.